

Lecture 4: Finding the MLE with iterated filtering

Qianying (Ruby) Lin Jesse Wheeler Spencer J. Fox

Objectives

By the end of this lecture you will be able to:

1. Say what the **maximum-likelihood estimate** is, and why a standard optimizer cannot find it on a noisy ridge.
2. Explain the idea behind iterated filtering, without pseudocode.
3. Run `mif2` on your laptop and read its output.
4. Say what each of the four knobs does, and what `cooling.fraction.50` actually means.
5. Explain why the likelihood `mif2` reports is **not** your likelihood.
6. Read a convergence plot the way an experienced modeller reads one.

The maximum-likelihood estimate

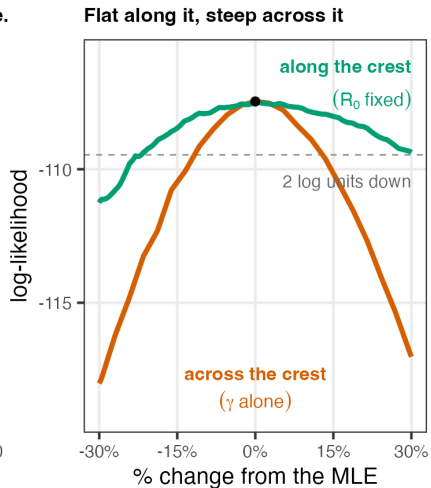
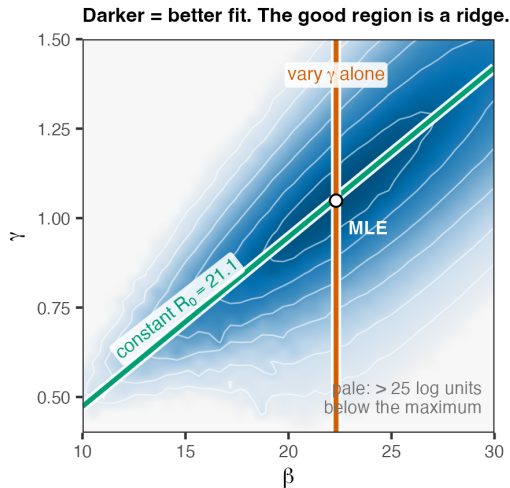
Before the break we built a machine that **scores** any fixed parameter set: `pfilter` turns a parameter vector into a log-likelihood.

Now we want the **best** parameters, the ones that make the data most likely: the **maximum-likelihood estimate**,

$$\hat{\theta} = \operatorname{argmax}_{\theta} \ell(\theta).$$

The obvious move is to hand our scoring machine to an optimizer (`optim`, Nelder-Mead) and let it climb. First: what does the surface look like?

The surface is a knife-edge ridge



The two **lines** on the left are the two **curves** on the right: *along* the ridge (green, R_0 fixed) the fit barely changes; *across* it (orange, γ alone, so R_0 changes) it drops three to five times faster.

The surface is a knife-edge ridge

- ▶ This is not a hill with a summit. It is a long diagonal **ridge**.
- ▶ The ridge runs along **constant** $R_0 = \beta/\gamma$.

The epidemiology, which matters more than the geometry:

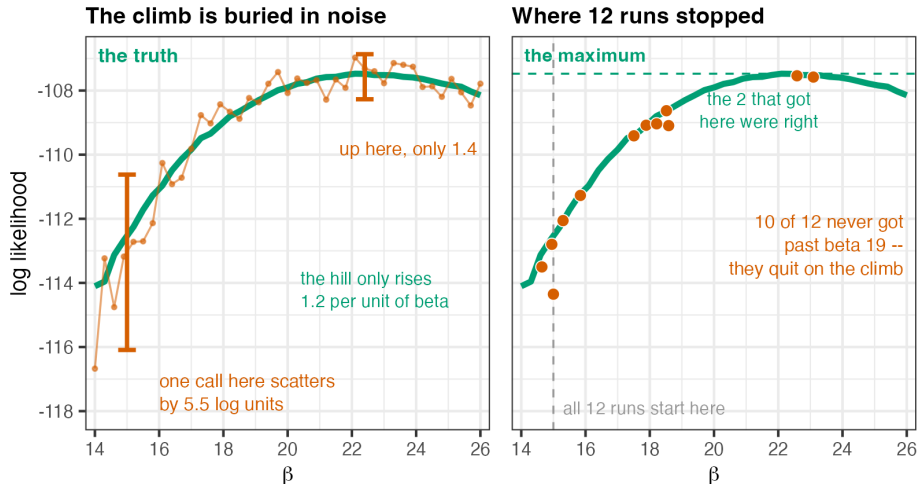
*These data pin down R_0 **sharply**. They barely constrain β and γ **separately**.*

You can trade a faster transmission rate against a faster recovery rate and the epidemic looks the same. The data cannot tell those apart, so neither can any method, no matter how good.

So what? R_0 is what these data actually measure. Report it with confidence; report β on its own and you are claiming precision that is not there. You will meet this ridge again in a moment, when twenty `mif2` searches agree on the fit but disagree about β by 50%.

And the surface is noisy

Every evaluation is a *different* random estimate. The optimizer does not see the smooth ridge. It sees this:



Why the obvious thing fails

Standard optimizers assume the function is:

- ▶ **cheap**: ours takes seconds per evaluation;
- ▶ **smooth**: ours is jagged with Monte Carlo noise;
- ▶ **deterministic**: ours gives a different answer every call.

They are built to walk downhill on a smooth landscape. Hand them a noisy one and they walk confidently into the noise and stop.

The optimizer isn't broken. The function you handed it is.

We need an algorithm built for this: iterated filtering.

Stop fighting the noise. Use it.

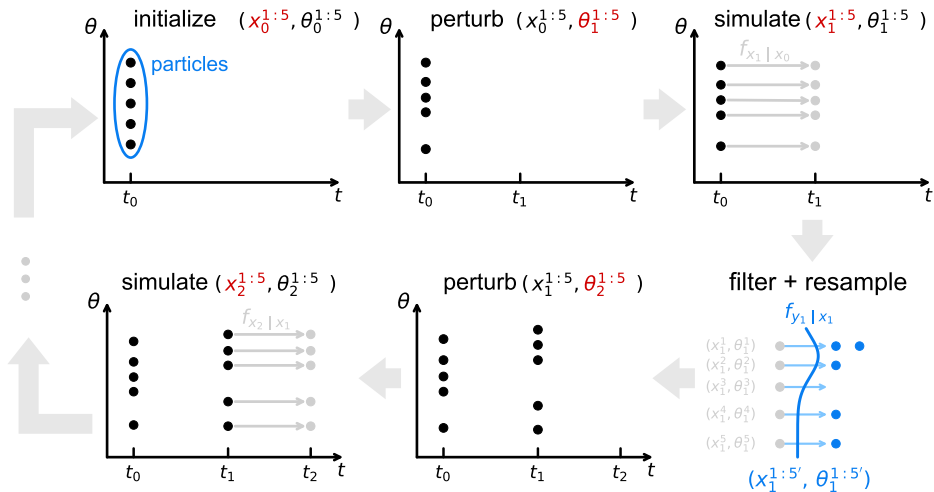
The particle filter already does something remarkable: it keeps a swarm of guesses about the **hidden state**, kills the bad ones, and clones the good ones.

Iterated filtering asks one question:

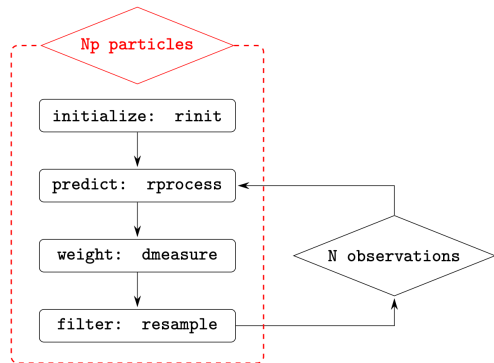
*What if we let the **parameters** wander too, and let the same selection process do the work?*

- ▶ Give every particle its own slightly different parameters.
- ▶ Run a particle filter. Particles that fit the data survive.
- ▶ Their parameters survive with them.
- ▶ Repeat, with a little less wandering each time.

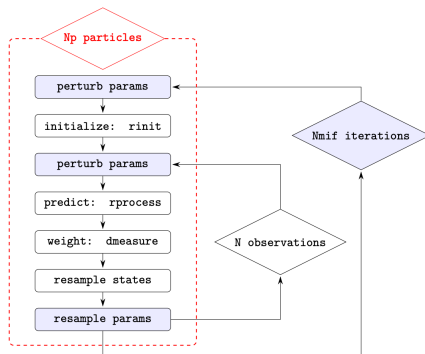
Iterated filtering, in one picture



mif2 is pfilter plus two boxes



before the break: pfilter



now: mif2

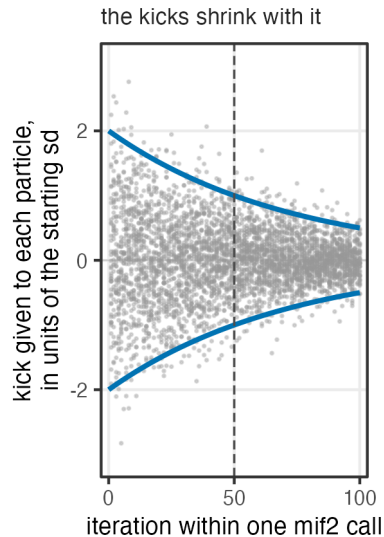
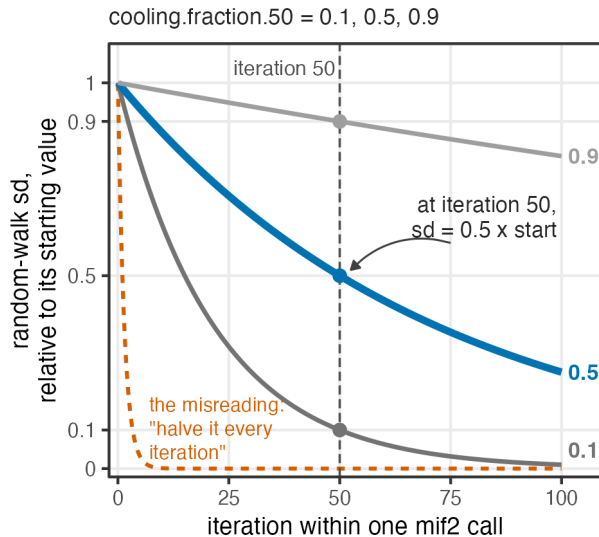
Same loop. Two new blue boxes: **perturb the parameters**, and **resample them along with the states**. Plus an outer loop.

The four knobs

```
po |> mif2(  
  Np=2000,           # particles  
  Nmif=50,           # iterations  
  cooling.fraction.50=0.5, # how fast perturbations shrink  
  rw.sd=rw_sd(  
    Beta=0.02, Rho=0.02, Eta=ivp(0.02)  
  )  
) -> mf  
  
plot(mf)
```

- ▶ `Np`, `Nmif`: how much compute you are willing to spend.
- ▶ `rw.sd`: how far parameters wander per step.
- ▶ `ivp()`: marks a parameter that only affects the *start* of the epidemic, so it is only perturbed there.

cooling.fraction.50 is a promise about iteration 50



`cooling.fraction.50` is a promise about iteration 50

`cooling.fraction.50=0.5` means:

*After **fifty** iterations, the perturbations are **half** the size they started at.*

It does **not** mean “halve them every iteration”, and it does not mean “halve them every fifty iterations”. The decay is smooth and continuous, and it keeps going after iteration 50.

- ▶ Cool too fast (0.1): the search freezes wherever it happens to be.
- ▶ Cool too slow (0.9): the search never settles.

Exercises 1 and 2. Open `4_mle_mif2.Rproj`, then `scripts/EXERCISES.R`.

One search tells you almost nothing

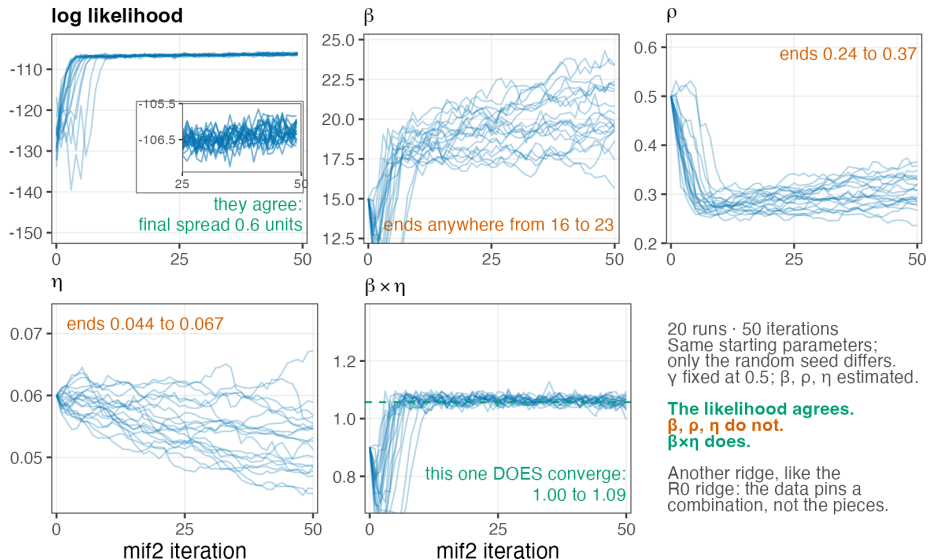
Run twenty. They are independent, so run them in parallel.

```
measSIR |> pomp(          # build the transform ONCE, outside the loop
  partrans=parameter_trans(log="Beta",logit=c("Rho","Eta")),
  paramnames=c("Beta","Rho","Eta")
) -> po

foreach(i=1:20, .combine=c, .packages="pomp", .export="po",
  .options.RNG=482947940) %dorng% {
  po |> mif2(Np=2000, Nmif=50, cooling.fraction.50=0.5,
    rw.sd=rw_sd(Beta=0.02, Rho=0.02, Eta=ivp(0.02))) -> mf
  mf
} -> mifs_local
```

Build `parameter_trans` **outside** the loop. Inside, it forces a recompile on every worker, a classic source of grief.

Twenty searches at once



The most important plot in this module

Look at those two facts together:

- ▶ The **log-likelihood** panel converges. All twenty searches agree the fit is good.
- ▶ The **parameter** panels do not. β disagrees by 50%.

Both are true at once, and **neither is a bug**.

The twenty answers are not a scattered cloud. They lie almost exactly on a **line**: higher transmission, spread through a smaller pool of susceptibles, with more of it reported, produces very nearly the same 42 numbers.

This is that same knife-edge ridge, now seen from the inside.

Two different questions

“Is my fit good?”

Read the log-likelihood panel.

Here: yes. All twenty agree.

“Do I know this parameter?”

Read that parameter's panel.

Here: **no**. Not β , not on its own.

A convergence plot answers both questions, and beginners routinely read only the first one.

Exercise 3. Run twenty searches and read the traces.

Still climbing? Run it longer

The healthy trace flattens. If your log-likelihood panel is **still rising** at the last iteration, the search has simply not finished.

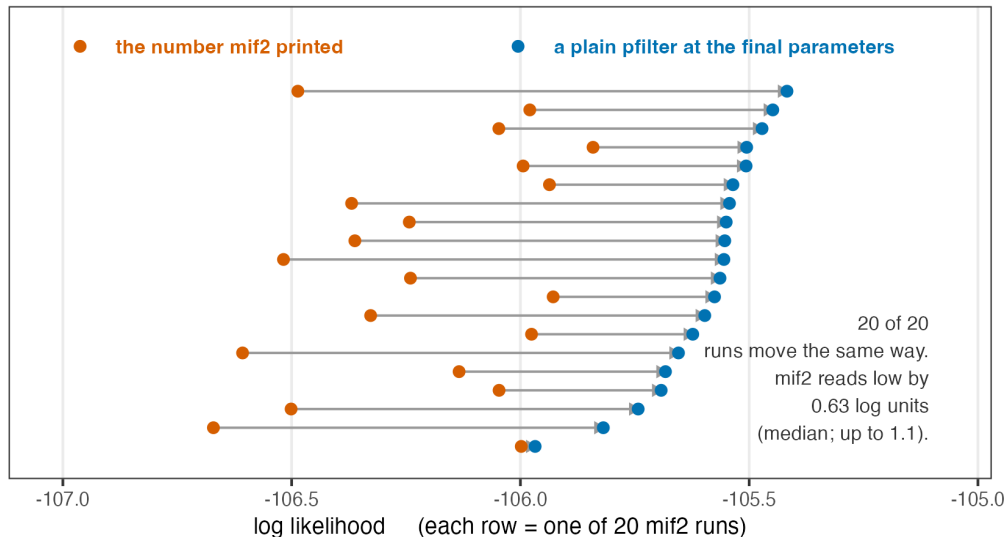
mif2 can be **continued**: hand a finished run back to mif2 and it re-runs from your current best parameters.

```
# within a foreach loop, or on a single run, we previously had:
po |> mif2(Np=2000, Nmif=50, cooling.fraction.50=0.5,
          rw.sd=rw_sd(Beta=0.02, Rho=0.02, Eta=ivp(0.02))) -> mf
# add this line to continue it with 50 more iterations:
mf |> mif2(Nmif=50) -> mf
```

- ▶ Continue each of the 20 runs, then re-plot the traces.
- ▶ Rule of thumb: if the loglik panel still trends upward at the end, add more iterations. (You met continuation in **Exercise 2**.)

Is the reported likelihood the true likelihood? NO!

- Always re-filter the final parameter vector to get a **TRUE** likelihood.



Why the reported number lies

Two reasons stack up:

1. During a `mif2` run the parameters are **deliberately perturbed**. The likelihood being reported belongs to a *noisier model than yours*.
2. `mif2` records the likelihood of the parameters at the **start** of each iteration. So your final parameter vector (the one in `coef(mf)`, the one you keep) has **never been filtered at all**.

Look at `traces(mf)`: the last log-likelihood is NA. That is not a bug. It is the point.

```
evals <- replicate(10, logLik(pfilter(mf, Np=5000)))  
logmeanexp(evals, se=TRUE)
```

Exercise 4. Re-estimate honestly, and compare.

Stalled or dropping? Diagnose, do not panic

A trace that **dips or wanders** is not automatically a failure. The number `mif2` reports is the likelihood of the **perturbed** model, estimated by a particle filter, so it is random and need not climb every step.

If it genuinely will not improve, check the usual culprits:

- ▶ **Too few particles** (`Np`): the estimate is noisy and biased low, and the filter may be losing particles. Raise `Np`, and check the effective sample size.
- ▶ **`rw.sd` too large**, or **cooling too slow**: the swarm never settles.

The verdict never comes from the trace. Re-filter the final parameters with `pfilter`, on your real, unperturbed model.

Reference: the `pomp` FAQ 5.2, <https://kingaa.github.io/pomp/FAQ.html>

The exercises at a glance

1. **Your first mif2.** Run one search; see what it changed.
2. **Turn the knobs.** Continue a run; feel what `cooling.fraction.50` does.
3. **Many searches at once.** Run 20; read the log-likelihood panel and the parameter panels as *different questions*.
4. **The reported number lies.** Re-filter each endpoint honestly, keep the best.
5. **(stretch) Does the start matter?** Scatter the starting points. Do they all arrive at the same place?
6. **(lunch break) Does the latent period earn its keep?** Fit the **SEIR** you built in Lecture 2 and compare it to your SIR. One extra parameter, worth it?

Why this algorithm, and not another

Many methods have been proposed for POMP models. Three questions sort them out:

- ▶ Does it need a **formula** for the process, or will simulation do?
- ▶ Does it use **all** the data, or just summaries?
- ▶ Does it need a **prior**?

Iterated filtering: simulation is enough, uses all the data, no prior required.

	Frequentist	Bayesian
Plug-and-play		
Full-information	iterated filtering	particle MCMC
Feature-based	simulated moments synthetic likelihood (SL) nonlinear forecasting	ABC SL-based MCMC
Not plug-and-play		
Full-information	EM algorithm Kalman filter	MCMC
Feature-based	Yule-Walker ¹ extended Kalman filter ²	extended Kalman filter ²

¹Yule-Walker is a method of moments for ARMA, a linear Gaussian POMP.

²The Kalman filter gives the exact likelihood for a linear Gaussian POMP. The extended Kalman filter gives an approximation for nonlinear models that can be used for quasi-likelihood or quasi-Bayesian inference.

That combination is why we teach it first (Ionides et al. 2015; King, Nguyen, and Ionides 2016).

Where we've got to

- ▶ Iterated filtering lets the **parameters** ride along in the particle filter, and lets selection improve them.
- ▶ Perturb, filter, resample, repeat, **cooling** the perturbations as you go.
- ▶ `cooling.fraction.50` is a promise about **iteration 50**.
- ▶ One search tells you nothing. **Run many.**
- ▶ The log-likelihood panel and the parameter panels answer **different questions**.
- ▶ The number `mif2` reports is not your likelihood. **Always re-filter.**

**Next, a full workflow: diagnostics, global searches,
and how to budget the compute.**

References

- Ionides, Edward L., Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. 2015. "Inference for Dynamic and Latent Variable Models via Iterated, Perturbed Bayes Maps." *Proc Natl Acad Sci* 112 (3): 719–24. <https://doi.org/10.1073/pnas.1410597112>.
- King, Aaron A., Dao Nguyen, and Edward L. Ionides. 2016. "Statistical Inference for Partially Observed Markov Processes via the R Package Pomp." *J Stat Softw* 69 (12): 1–43. <https://doi.org/10.18637/jss.v069.i12>.

License, acknowledgments, and links

- ▶ This lesson is prepared for the Simulation-based Inference for Epidemiological Dynamics module at the Summer Institute in Statistics and Modeling in Infectious Diseases, SISMID.
- ▶ The materials build on previous versions of this course and related courses.
- ▶ Licensed under the Creative Commons Attribution-NonCommercial license. Please share and remix non-commercially, mentioning its origin. 
- ▶ Produced with R version 4.5.1 and pomp version 6.4.

R code for this lesson

The maximum-likelihood estimate

For reference: the same ideas as the lecture's pictures, written as the formulas from the original *iterated filtering* lesson. None of this is needed for the exercises.

Lecture A learned to *evaluate* the likelihood at a fixed θ , $\mathcal{L}(\theta) = f_{Y_{1:N}}(y_{1:N}^*; \theta)$. This lecture *maximises* it. The **maximum-likelihood estimate** is

$$\hat{\theta} = \operatorname{argmax}_{\theta} \ell(\theta), \quad \ell(\theta) = \log \mathcal{L}(\theta).$$

Two facts make this hard, and both shape the algorithm. Each value $\ell(\theta)$ is a **noisy, expensive** particle-filter estimate, and the surface is a near-flat **ridge** along constant $\mathfrak{R}_0 = \beta/\gamma$. So IF2 never differentiates ℓ and never trusts a single evaluation: it is a stochastic-approximation search built for exactly this surface.

Iterated filtering (IF2): inputs and knobs

Input. Simulators for the initial and transition densities $f_{X_0}(x_0; \theta)$ and $f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta)$; an evaluator for the measurement density $f_{Y_n|X_n}(y_n | x_n; \theta)$; and the data $y_{1:N}^*$. It calls `rprocess` and `dmeasure` only, never `dprocess`: IF2 is a **plug-and-play** method.

Algorithmic parameters (with their `mif2` arguments):

- ▶ `Nmif = M`: number of filtering iterations.
- ▶ `Np = J`: number of particles.
- ▶ initial parameter swarm $\{\Theta_j^0\}_{j=1}^J$.
- ▶ `rw.sd`: per-parameter random-walk sd, squared into a diagonal variance matrix V_n .
- ▶ `cooling.fraction.50 = a`: how fast the perturbations shrink.

Cooling. At iteration m the perturbation variance is $V_n a^{2m/50}$, so the standard deviation carries $a^{m/50}$. Thus $a = 0.5$ halves the sd by iteration 50, quarters it by 100, and so on: smooth, continuous, and never quite zero. **Output:** the final swarm $\{\Theta_j^M\}$, whose mean or median is $\hat{\theta}$.

The IF2 algorithm I

Procedure.

1. For m in $1:M$:
2. *perturb*: $\Theta_{0,j}^{F,m} \sim \text{Normal}(\Theta_j^{m-1}, V_0 a^{2m/50})$, $j = 1:J$
3. *initialise*: $X_{0,j}^{F,m} \sim f_{X_0}(x_0; \Theta_{0,j}^{F,m})$, $j = 1:J$
4. For n in $1:N$:
5. *perturb*: $\Theta_{n,j}^{P,m} \sim \text{Normal}(\Theta_{n-1,j}^{F,m}, V_n a^{2m/50})$
6. *predict*: $X_{n,j}^{P,m} \sim f_{X_n|X_{n-1}}(x_n | X_{n-1,j}^{F,m}; \Theta_{n,j}^{P,m})$
7. *weight*: $w_{n,j}^m = f_{Y_n|X_n}(y_n^* | X_{n,j}^{P,m}; \Theta_{n,j}^{P,m})$
8. *resample*: draw $k_{1:J}$ with $\mathbb{P}[k_j = i] = w_{n,i}^m / \sum_{u=1}^J w_{n,u}^m$
9. set $\Theta_{n,j}^{F,m} = \Theta_{n,k_j}^{P,m}$ and $X_{n,j}^{F,m} = X_{n,k_j}^{P,m}$
10. End For
11. Set $\Theta_j^m = \Theta_{N,j}^{F,m}$, $j = 1:J$
12. End For

The IF2 algorithm II

Reading the algorithm.

- ▶ The inner loop (lines 4–10) is exactly the particle filter of Lecture A, run on a model whose parameters get a fresh perturbation every step.
- ▶ The outer loop (lines 1–12) repeats that filter M times, shrinking the perturbations each pass. That shrinkage is the cooling.
- ▶ Superscript F marks the *filtering* distribution (particles given $y_{1:n}^*$); superscript P marks the *prediction* distribution (given $y_{1:n-1}^*$).
- ▶ The weight $w_{n,j}^m$ is the likelihood of observation n under particle j : fitter particles are resampled more often, and their perturbed parameters ride along.
- ▶ As $J \rightarrow \infty$ with a suitable cooling schedule, the swarm concentrates on $\hat{\theta}$ (Ionides et al. 2015). IF2 is the only current **full-information, plug-and-play, frequentist** method for POMP models.